Lab5

Course: ARTI 404 – Image Processing

Student Name: Muhannad Almutlaq

Student ID: 2240006060

Task#1

Convolve an image with a 7x7 box filter, and output the original and smoothed image side by side.

```
image= cv2.imread(Path("./images/image1.png"))
image= cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert BGR to RGB

# Kernel Size: (7x7) box filter
smoothed_image= cv2.blur(image, (10, 10))

fig, ax = plt.subplots(1, 2, figsize= (12, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(smoothed_image)
ax[1].set_title("Smoothed Image (7x7) Box Filter")
ax[1].axis("off")

plt.show()
```



Task#2

Using an image of your choice, apply a 5x5 Gaussian filter to the image, then a 21x21 Gaussian filter, and show the three images side by side.

```
image= cv2.imread(Path("./images/image1.png"))
image= cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Kernel Size: (5x5) Gaussian filter to the image
gaussian_5x5= cv2.GaussianBlur(image, (5, 5), 0)

# Kernel Size: (21x21) Gaussian filter
gaussian_21x21 = cv2.GaussianBlur(image, (21, 21), 0)

# Show the three images side by side
fig, ax = plt.subplots(1, 3, figsize= (15, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(gaussian_5x5)
ax[1].set_title("(5x5) Gaussian Filter")
ax[1].axis("off")

ax[2].imshow(gaussian_21x21)
ax[2].set_title("(21x21) Gaussian Filter")
ax[2].axis("off")

plt.show()
```



Task#3

Using an image of your choice, apply a 3x3 Laplacian filter to sharpen the image. Follow the following steps:

1. Read the image
2. Convert image to float32
3. Use the cv2.Laplacian() function
4. Sharpen the image using: np.clip(image_float - laplacian, 0, 1)
5. Plot the original image, the resulting image after applying the Laplacian filter, and the sharpened image

```
image= cv2.imread(Path("./images/image2.png"))
image= cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to float32
image_float= image.astype(np.float32) / 255.0

laplacian= cv2.Laplacian(image_float, cv2.CV_32F, ksize= 3) # Using cv2.CV_32F
destination depth to maintain the float32 format and handle negative values.

laplacian_display= np.clip(np.abs(laplacian), 0, 1) # Take the absolute value to map
negative gradients to bright edges, then clip to [0, 1].

sharpened= np.clip(image_float - laplacian, 0, 1)

fig, ax = plt.subplots(1, 4, figsize= (15, 5))

ax[0].imshow(image_float)
ax[0].set_title("Original Image")
ax[0].axis("off")

ax[1].imshow(laplacian)
ax[1].set_title("Laplacian Filter (3x3)")
ax[1].axis("off")

ax[2].imshow(laplacian_display)
ax[2].set_title("Laplacian Filter (3x3) clipping [0, 1] positives")
ax[2].axis("off")

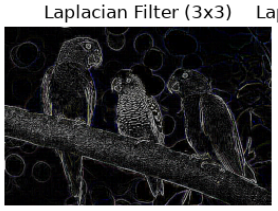
ax[3].imshow(sharpened)
ax[3].set_title("Sharpened Image")
ax[3].axis("off")

plt.show()
```

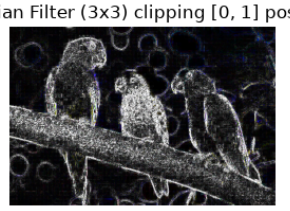
Original Image



Laplacian Filter (3x3)



Laplacian Filter (3x3) clipping [0, 1] positives



Sharpened Image

